Matrix Notation

- $A \in \mathbb{R}^{m \times n}$: Matrix with m rows and n columns
- A_{ij} or $(A)_{ij}$: Element in row i, column j of matrix A
- A^T: Transpose of matrix A
- A^{-1}: Inverse of matrix A (when it exists)
- $\text{tr}(A)$: Trace of matrix A (sum of diagonal elements)
- $\det(A)$ or $|A|$: Determinant of matrix A
- $\text{rank}(A)$: Rank of matrix A
- I or I_n: Identity matrix of size $n \times n$

1.3 Probability and Statistics

Probability Notation

- $P(A)$: Probability of event A
- $P(A|B)$: Conditional probability of A given B
- $P(A, B)$ or $P(A \cap B)$: Joint probability of A and B
- $X \sim \mathcal{N}(\mu, \sigma^2)$: Random variable X follows normal distribution
- $X \sim \text{Bernoulli}(p)$: X follows Bernoulli distribution
- $\mathbb{E}[X]$: Expected value (mean) of random variable X
- $\text{Var}(X)$: Variance of random variable X
- $\text{Cov}(X, Y)$: Covariance between random variables X and Y
- $f(x)$ or $p(x)$: Probability density function (continuous)
- $P(x)$ or $p(x)$: Probability mass function (discrete)

1.4 Machine Learning Notation

Training Data

- n or m: Number of training examples
- d: Number of features (dimension)
- $(x^{(i)}, y^{(i)})$: The i-th training example
- $X \in \mathbb{R}^{n \times d}$: Design matrix (training examples as rows)
- $y \in \mathbb{R}^n$: Vector of target values
- $\mathcal{D} = \{(x^{(1)}, y^{(1)}), \ldots, (x^{(n)}, y^{(n)})\}$: Training dataset

Big-O

Big-O Notation Symbols

- $O(f(n))$: Upper bound (at most)
- $\Omega(f(n))$: Lower bound (at least)
- $\Theta(f(n))$: Tight bound (exactly)
- $o(f(n))$: Strict upper bound (strictly less than)
- $\omega(f(n))$: Strict lower bound (strictly greater than)

How to Use Big-O Notation

Time Complexity Examples:

- Matrix multiplication: $O(n^3)$ for $n \times n$ matrices
- Gradient descent (one iteration): $O(nd)$ where n = samples, d = features
- Training SVM: $O(n^3)$ in worst case, $O(n^2)$ typical
- K-means (one iteration): $O(nkd)$ where k = clusters

Space Complexity Examples:

- Storing training data: $O(nd)$
- Neural network parameters: $O(L \cdot H^2)$ where L = layers, H = hidden units
- Kernel matrix for SVM: $O(n^2)$

Reading Big-O:

- "Algorithm A is $O(n^2)$" means: for large n, runtime grows at most quadratically
- "Space requirement is $O(n)$" means: memory usage grows linearly with input size
- Constants are ignored: $O(5n + 100) = O(n)$

Formulas, Notations and important notations

1. Matrix Derivatives

Expression	Gradient ∇_x	Hessian ∇_x^2	Key intuition
$\frac{1}{2} x^\top A x$ (A symmetric)	$A x$	A	Quadratic form is a "paraboloid" whose curvature equals A.
$b^\top x$	b	0	Linear tilt; no curvature.
$g(h(x))$ (scalar→scalar)	$g'(h(x))\, \nabla_x h(x)$	_	Scalar chain rule plus inner gradient.
$g(a^\top x)$	$g'(a^\top x)\, a$	$g''(a^\top x)\, a a^\top$	Output only changes along the direction a.
Log-sum-exp $\log \sum_i e^{x_i}$	$\mathrm{softmax}(x)$	Diagonal − outer-product of softmax	Softmax is gradient of LSE; Hessian is PSD → convex.

Mnemonic: treat every operation as *scalar* first, then "re-vectorize."

Reinforcement Learning

1. The World: Markov Decision Process (MDP) 🗺️

This is the formal description of the environment.

- **States (s)**: A set of distinct situations the agent can be in (e.g., squares on a chessboard).
- **Actions (a)**: A set of choices the agent can make from a state (e.g., move piece to a new square).
- **Transition Model T(s, a, s')**: The rules of the world. It gives the probability P(s' | s, a) of landing in state s' after taking action a in state s.
- **Reward Function R(s, a, s')**: The feedback from the world. It gives the reward r received after transitioning from s to s' via action a.

---

2. The Goal: Finding the Optimal Policy (π*) 🏆

- **Policy (π)**: The agent's strategy or "brain," which maps states to actions: π(s) -> a.
- **Optimal Policy (π*)**: The policy that maximizes the total expected future discounted rewards.
- **The Core Dilemma: Exploration vs. Exploitation**
  - **Exploitation**: Sticking with the action that currently seems best.
  - **Exploration**: Trying a new, random action to see if it might be even better.
  - **Solution (e.g., ε-greedy)**: With probability (1-ε), pick the best-known action; with probability ε, pick a random one.

---

3. Measuring "Goodness": Value Functions & Discounting

We need a way to measure the long-term value of being in a state or taking an action.

- **Discount Factor (γ)**: A number between 0 and 1. It makes future rewards less valuable than immediate ones. A γ of 0.9 means a reward 10 steps away is only worth $0.9^{10} \approx 0.35$ times as much as a reward right now.
- **State-Value Function V(s)**: The total expected future discounted reward starting from state s and then following a policy π. "How good is it to be in this state?"
- **Action-Value Function Q(s, a)**: The total expected future discounted reward from first taking action a in state s, and then following a policy π. "How good is it to take this action in this state?"

---

4. The Rulebook: The Bellman Equations 🧠

These equations connect the value of a state to the value of its successor states. They are the foundation for all learning.

- Bellman Equation for V(s):
  V(s) = E[Immediate Reward + γ * V(next state)]
- Bellman Equation for Q(s, a):
  Q(s, a) = E[Immediate Reward + γ * Q(next state, next action)]

---

5. The Algorithms: How We Actually Learn

The choice of algorithm depends on whether we know the "map" of the world (T and R).

Setting	**Model-Based** (You have the map)	**Model-Free** (You DON'T have the map)
How it Works	You can plan and compute the optimal policy without ever taking a step.	You must learn by taking steps and observing the outcomes (trial and error).
Key Algorithm	**Value Iteration**	**Q-Learning**
Update Rule	It repeatedly sweeps through all states, updating the **Value** of each state based on its neighbors: $V(s) \leftarrow \max_a \Sigma T(s,a,s')[R(...) + \gamma V(s')]$	It updates the **Q-Value** of a single state-action pair based on one experience (s, a, r, s'): $Q(s,a) \leftarrow Q(s,a) + \alpha * [\text{Target} - Q(s,a)]$

Model-Free Learning Targets: The Two Scenarios

The core of Q-learning is the **Target** used in its update rule.

Scenario	1. **To Learn the Optimal Policy** (Q-Learning)	2. **To Evaluate a Fixed Policy (π)** (SARSA-like)
Goal	Find the absolute best path, regardless of current policy.	Find out how good a specific, given policy is.
The "Target" is...	$r + \gamma * \max_{a'} Q(s', a')$	$r + \gamma * Q(s', a')$ where $a' = \pi(s')$
In Words	The immediate reward (r) + the discounted value (γ) of the **best possible next action** (max Q).	The immediate reward (r) + the discounted value (γ) of the **next action the policy tells you to take**.

EM Algorithm for GMM

Repeat until convergence: {

(E-step) For each i, j, set

r_{ij} → $$w_j^{(i)} := p(z^{(i)} = j | x^{(i)}; \phi, \mu, \Sigma)$$

(M-step) Update the parameters:

π_j →

$$\phi_j := \frac{1}{n} \sum_{i=1}^{n} w_j^{(i)},$$

$$\mu_j := \frac{\sum_{i=1}^{n} w_j^{(i)} x^{(i)}}{\sum_{i=1}^{n} w_j^{(i)}},$$

$$\Sigma_j := \frac{\sum_{i=1}^{n} w_j^{(i)} (x^{(i)} - \mu_j)(x^{(i)} - \mu_j)^T}{\sum_{i=1}^{n} w_j^{(i)}}$$

}